Dimensionality Reduction

PCA, t-SNE and UMAP: fewer numbers per point, same structure

PCA

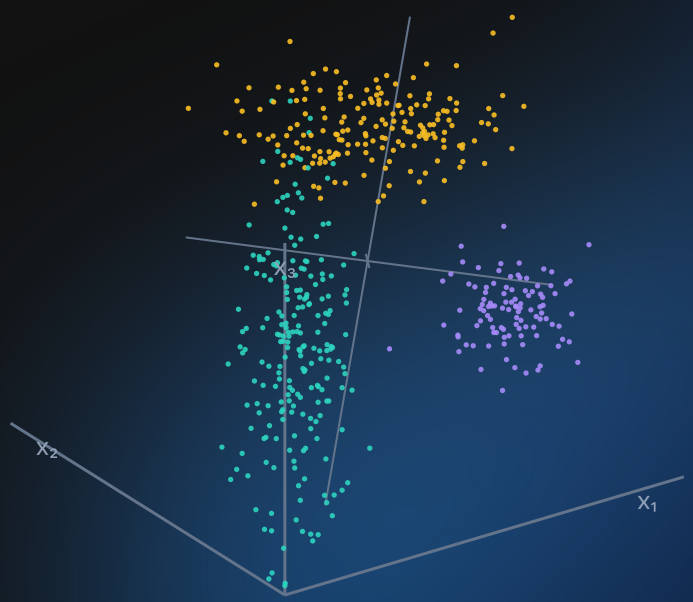
t-SNE

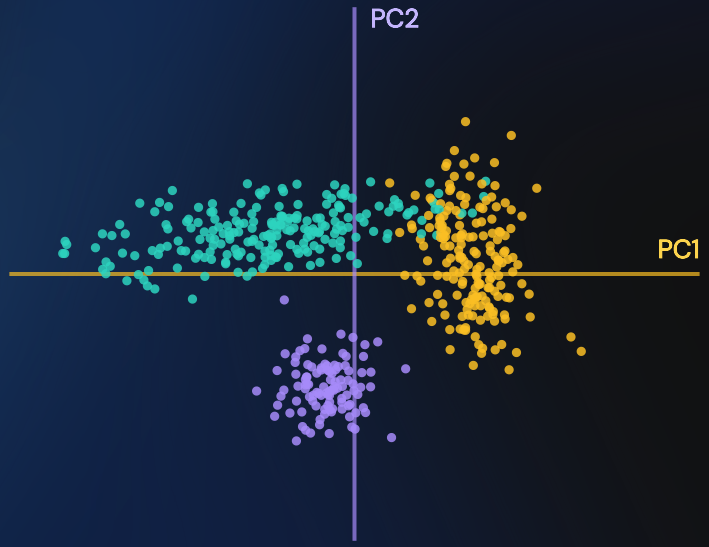
UMAP

64 pixels per digit → 2 coordinates

Unsupervised Learning · Foundations of Machine Learning

Where Gaussian Mixtures Left Off





Three numbers per point, but PC1 and PC2 hold **99.7%** of the variance. Two coordinates lose almost nothing.

Today

1 · Why fewer dimensions

Images as vectors, intrinsic dimension, the curse of dimensionality.

2 · PCA: the best flat view

Max variance = min error = top eigenvectors. Power iteration, SVD, scaling.

3 · PCA on real images

Eigendigits, reconstruction, choosing k, denoising, speeding up a model.

4 · When flat is not enough

Manifolds, the Swiss roll, and why only small neighborhoods can be trusted.

5 · t-SNE

Neighbor probabilities, heavy tails, KL divergence, live, and its pitfalls.

6 · UMAP, and putting it together

Fuzzy graphs, two knobs, comparisons, a practical pipeline.

Part 1 · Why Fewer Dimensions?

Data often has many columns but few real degrees of freedom.

An Image Is a Vector

$$x = (\ 0,\ 0,\ 7,\ 15,\ 13,\ 1,\ 0,\ 0,\ 0,\ 8,\ 13,\ 6,\ \ldots\ ,\ 13,\ 9,\ 0,\ 0\ ) \in \mathbb{R}^{64}$$

1797 scanned digits, 8 × 8 pixels each

64 features: one per pixel, ink from 0 to 16

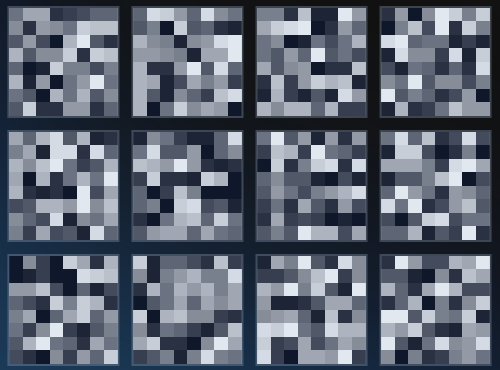
Each image is **one point** in $\mathbb{R}^{64}$

Most of $\mathbb{R}^{64}$ Is Not Digits

12 real digits

↻ Draw again

12 random points of $\mathbb{R}^{64}$

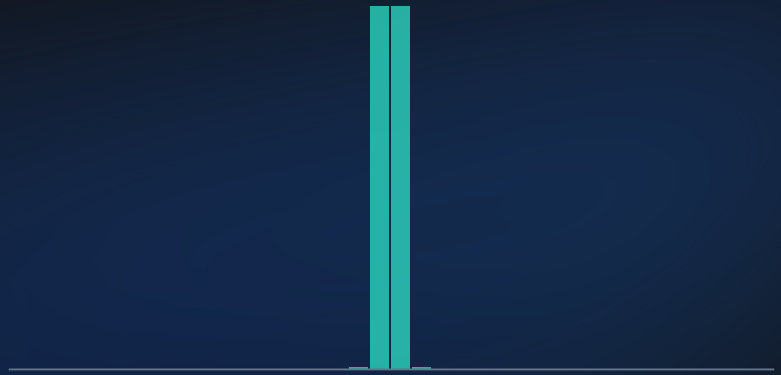


Intrinsic dimension: real images vary along a few directions: stroke thickness, slant, which digit. They fill a thin sliver of the space.

On digits, PCA needs **5** directions for 50% of the variance, **21** for 90%, **29** for 95%: not 64.

The Curse of Dimensionality

d = 2 · (max - min) / min = 272.69



d = 1000 · (max - min) / min = 0.15

Four Reasons to Reduce

👁️ Visualize

We can see 2 or 3 dimensions. Map the data there and look for clusters, outliers, trends.

🧹 Denoise

Noise spreads over all directions; signal concentrates in a few. Keep those, drop the rest.

⚡ Accelerate

Fewer features: faster training and neighbor search, fewer parameters to overfit.

🗜️ Compress

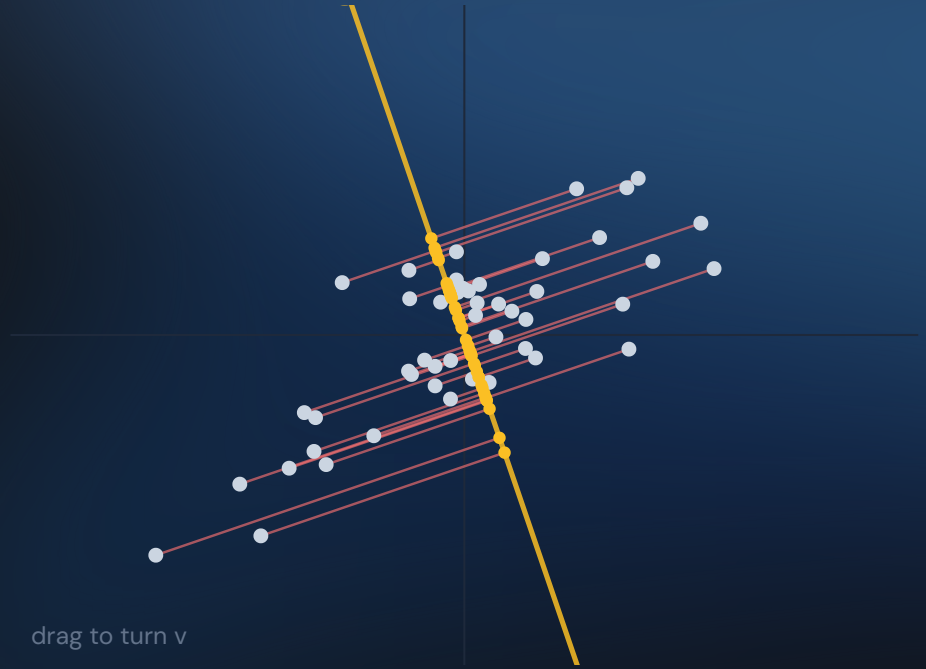
Store k numbers per item instead of d, and rebuild an approximation when needed.

Different goals, different methods: PCA does all four. t-SNE and UMAP are for the first one.

Part 2 · PCA: The Best Flat View

Which direction should we look along?

Projecting onto a Direction

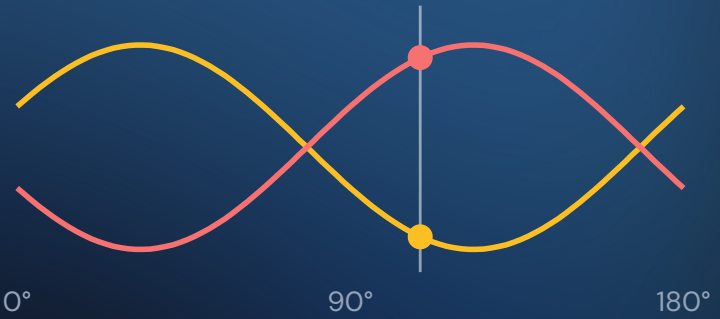


drag to turn v

angle 109°

kept $v^\top \Sigma v$ = 0.34 lost 2.05

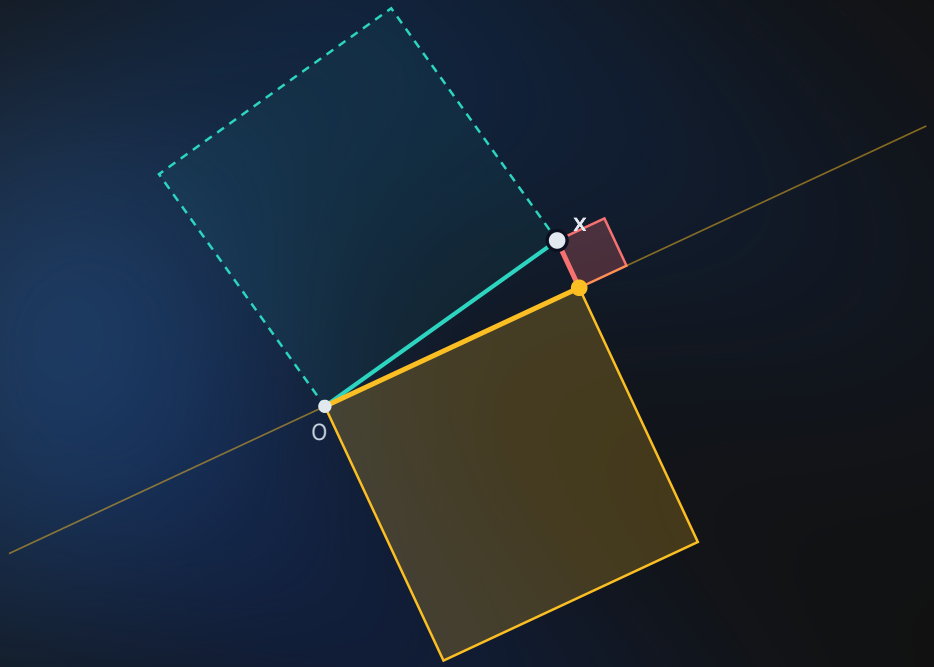
total variance tr Σ = 2.39, whatever v is



Snap to PC1 · λ_1 = 2.17

$$z_i = \mathbf{v}^\top \mathbf{x}_i \;\; (\|\mathbf{v}\| = 1,\ \text{data centered}), \qquad \mathrm{Var}(z) = \frac{1}{n-1}\sum_i (\mathbf{v}^\top \mathbf{x}_i)^2 = \mathbf{v}^\top \Sigma\, \mathbf{v}$$

Two Views, One Answer



$|x|^2 = 11.84$ fixed

$(v^\top x)^2 = 11.44$ kept

$|r|^2 = 0.40$ lost

$$\underbrace{\sum_i \|\mathbf{x}_i\|^2}_{\text{fixed}} = \underbrace{\sum_i (\mathbf{v}^\top \mathbf{x}_i)^2}_{\text{kept: maximize}} + \underbrace{\sum_i \|\mathbf{x}_i - (\mathbf{v}^\top \mathbf{x}_i)\mathbf{v}\|^2}_{\text{lost: minimize}}$$

The Optimizer Is an Eigenvector

Constrain — without $\|v\| = 1$, making v longer would make $v^T\Sigma v$ as big as we like.

Set the gradient to zero — $\nabla(v^T\Sigma v) = 2\Sigma v$, $\nabla(v^Tv) = 2v$.

Evaluate — at an eigenvector, the variance is its eigenvalue. Take the largest.

Repeat — best direction orthogonal to v_1 is v_2, and so on: the eigenvectors in order of λ.

$$\max_{\mathbf{v}} \ \mathbf{v}^\top \Sigma \mathbf{v} \quad \text{s.t.} \quad \mathbf{v}^\top \mathbf{v} = 1$$

$$\mathcal{L} = \mathbf{v}^\top \Sigma \mathbf{v} - \lambda(\mathbf{v}^\top \mathbf{v} - 1) \ \Rightarrow \ \Sigma \mathbf{v} = \lambda \mathbf{v}$$

$$\mathbf{v}^\top \Sigma \mathbf{v} = \mathbf{v}^\top (\lambda \mathbf{v}) = \lambda$$

$$\Sigma = Q \Lambda Q^\top, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$$

Seeing $\Sigma v = \lambda v$

```
angle(v, Σv) = 23.2°
vᵀΣv = 0.221
```

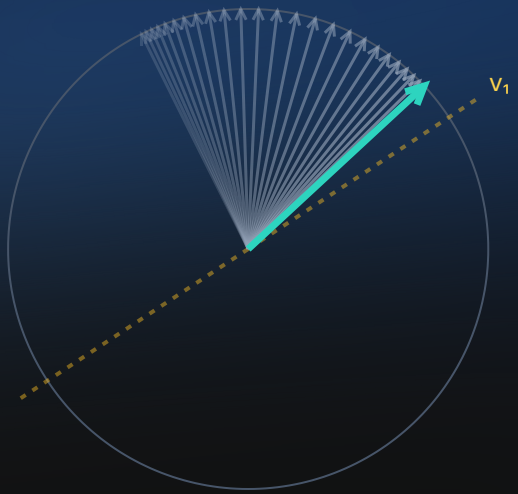
Drag v around the circle.

not an eigenvector
Σv points somewhere else

$\lambda_1 = 2.170$, $\lambda_2 = 0.216$: the largest and smallest values $v^T\Sigma v$ can take.

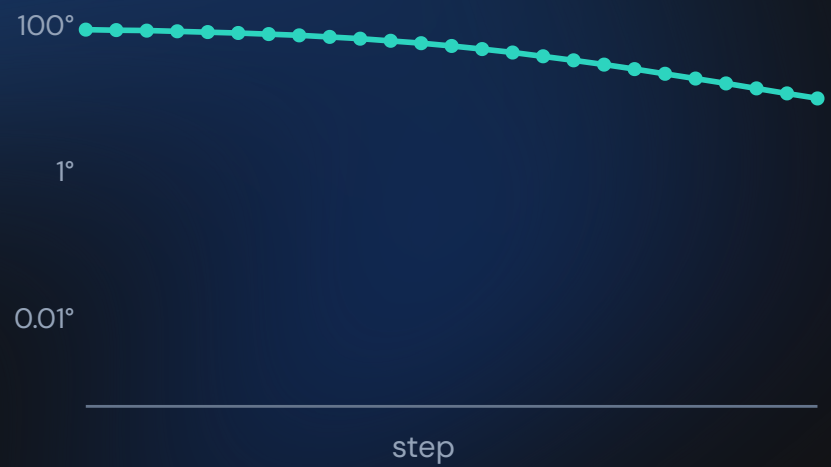
Finding v_1: Power Iteration

step 24



our cloud · λ_2/λ_1 = 0.10

close eigenvalues · λ_2/λ_1 = 0.85



angle to v_1: 9.5°

$$\mathbf{v} \leftarrow \frac{\Sigma \mathbf{v}}{\|\Sigma \mathbf{v}\|}$$ error shrinks by about λ_2/λ_1 per step

PCA Is a Change of Basis

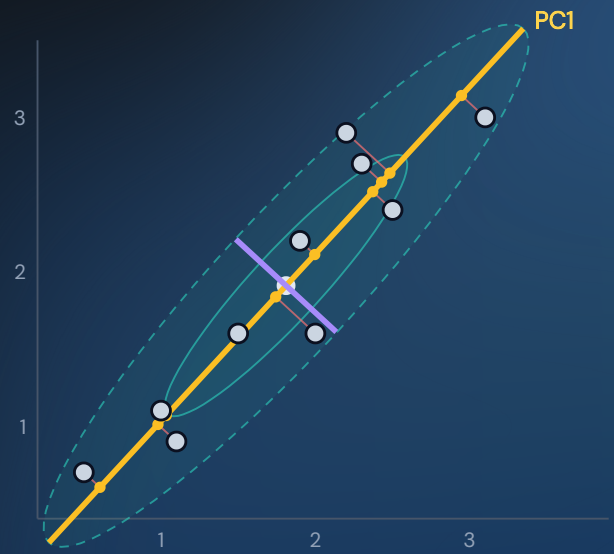
1 · Center
subtract the mean μ

2 · Rotate
Z = (X − μ)Q: eigenvectors become the axes

3 · Truncate
keep the first k columns of Z

4 · Reconstruct
$\hat{X} = \mu + Z_k Q_k^T$, error = dropped variance

Worked Example: PCA by Hand



10 points (Smith, 2002) · hover a point

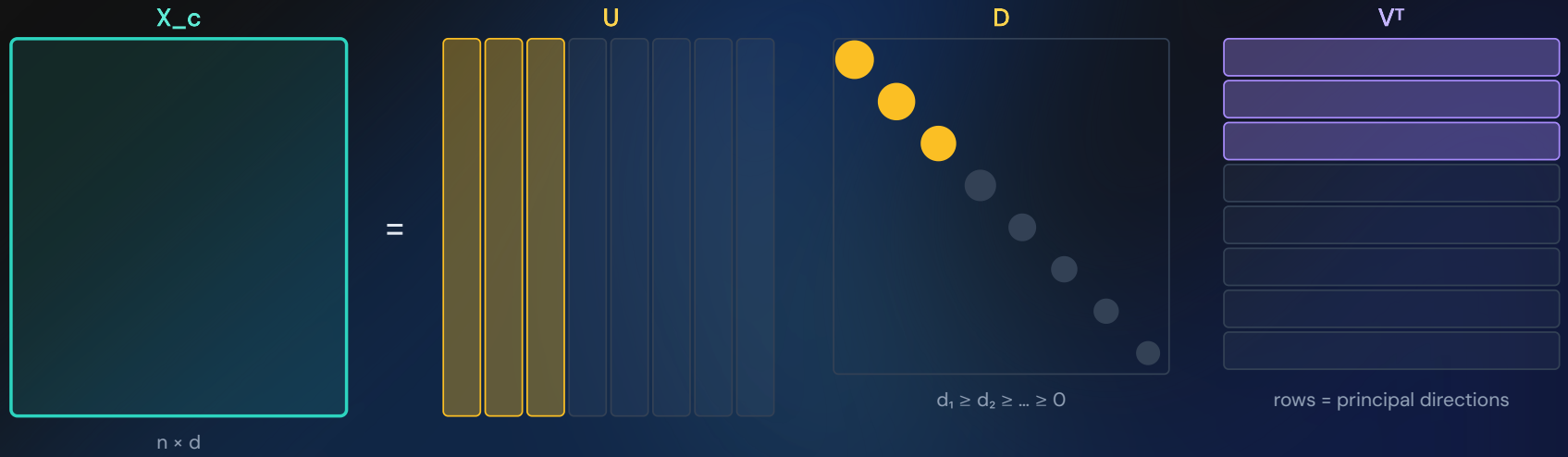
Mean $\bar{\mathbf{x}} = (1.81,\ 1.91)$, so center first.

$$\Sigma = \begin{bmatrix} 0.617 & 0.615 \\ 0.615 & 0.717 \end{bmatrix}$$

$$\det(\Sigma - \lambda I) = 0 \ \Rightarrow\ \lambda_1 = 1.284,\ \lambda_2 = 0.049, \quad \mathbf{v}_1 = (0.678,\ 0.735)$$

PC1 keeps $\lambda_1/(\lambda_1 + \lambda_2) =$ **96.3%** of the variance. Each point's score: $z_i = \mathbf{v}_1^\top(\mathbf{x}_i - \bar{\mathbf{x}})$.

The SVD Route, and What sklearn Does



k = 3

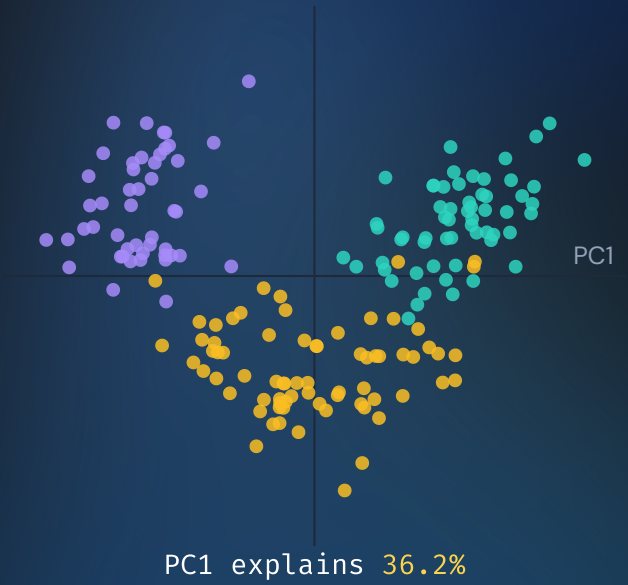
digits: first 3 keep 40.3% of the variance

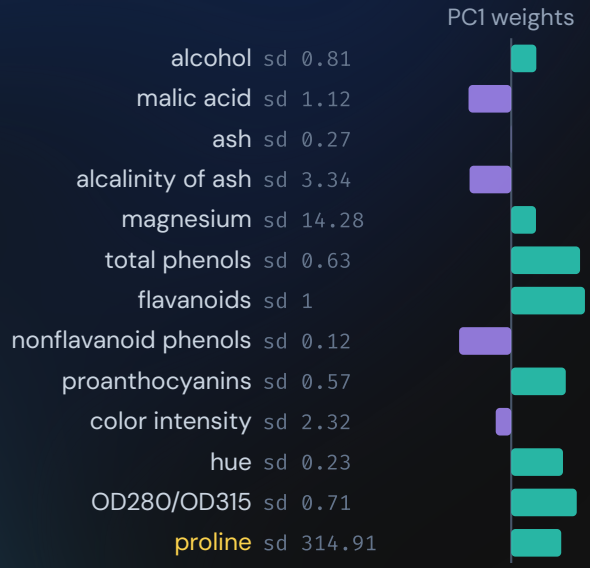
$$X_c = UDV^\top$$

$$\Sigma = \tfrac{1}{n-1} X_c^\top X_c = V \tfrac{D^2}{n-1} V^\top, \quad \lambda_j = \tfrac{d_j^2}{n-1}$$

`svd_solver="auto"`: tall data (n ≥ 10d, d ≤ 1000, like digits) → form Σ and eigendecompose it (`covariance_eigh`). Otherwise → SVD of X_c, full or randomized.

Scale Before PCA





Standardized: every feature has variance 1, so PC1 mixes many of them, and the three cultivars separate.

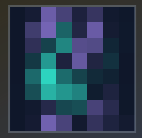
Part 3 · PCA on Real Images

64 pixels, 1797 digits: what do the components look like?

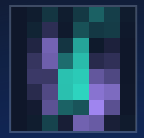
Eigendigits

mean μ

+ weights on…



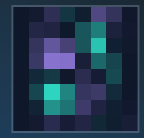
PC1 · 14.9%



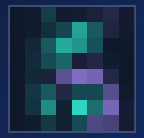
PC2 · 13.6%

PC3 · 11.8%

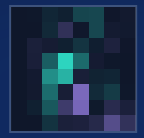
PC4 · 8.4%



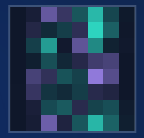
PC5 · 5.8%



PC6 · 4.9%

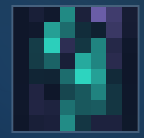


PC7 · 4.3%



PC8 · 3.7%

PC9 · 3.4%

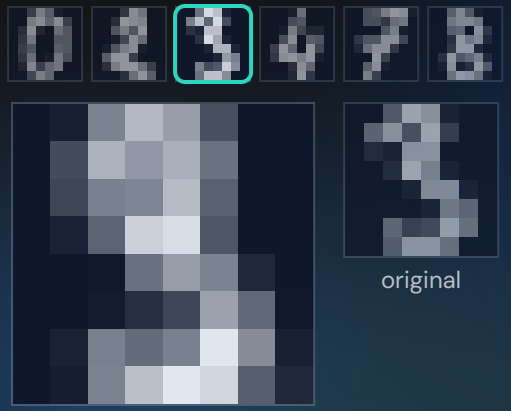


PC10 · 3.1%

μ + 2.0σ · PC1

hover a component

Rebuilding a Digit



k = 5

▶ build

-15.9× -3.3× -9.8× +12.3× + …

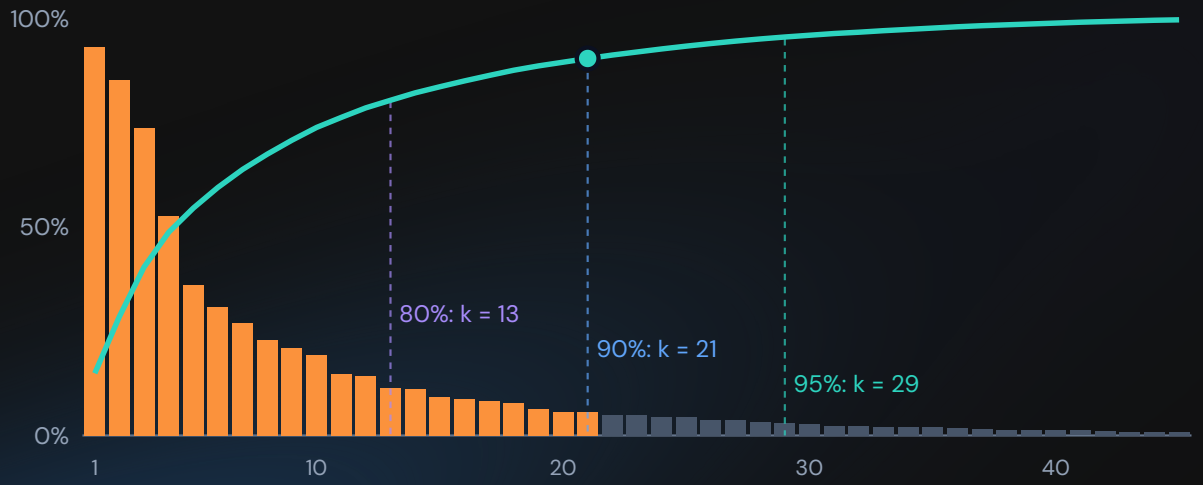
|z₁| |z₄₀|

variance kept (all digits): 54.5%

this digit's error ‖x - x̂‖²/64: 6.20

$$\hat{\mathbf{x}} = \boldsymbol{\mu} + \sum_{j=1}^{k} z_j \, \mathbf{v}_j, \qquad z_j = \mathbf{v}_j^{\top}(\mathbf{x} - \boldsymbol{\mu})$$

How Many Components?



k = 21 · 90.3% kept bars: each component (16% = full height) line: cumulative

The Digits on Two Components

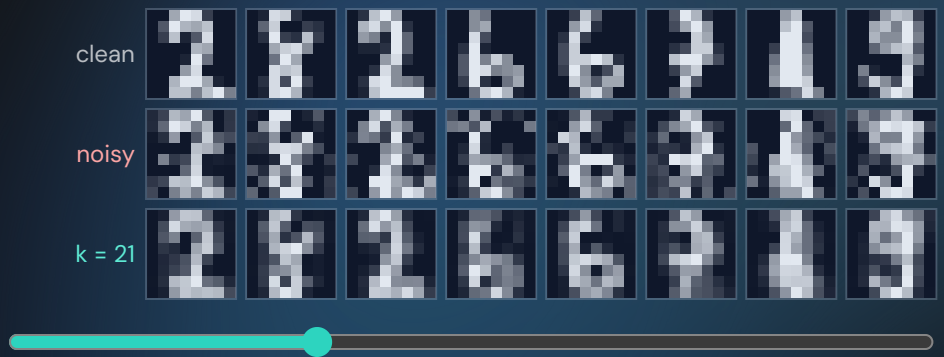
0
1
2
3
4
5
6
7
8
9

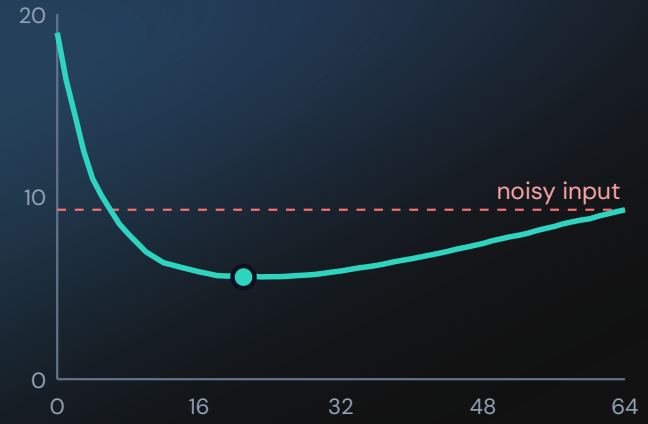
PC1 + PC2 hold only 28.5% of the variance.

0, 4, 6 and 2 sit apart; **1, 5, 8, 9** pile up in the middle.

Hover a point to see its image. Hover a digit in the legend to pick out its class.

PCA Denoises

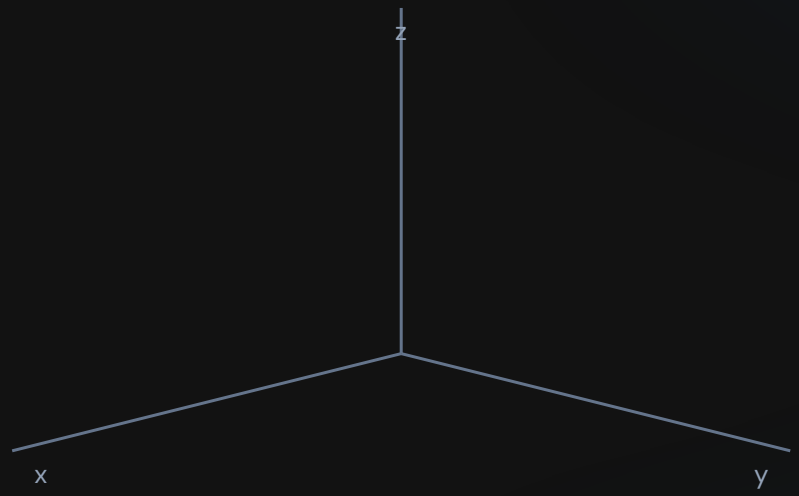




error 5.59 · noisy 9.28 · best k = 21

Signal lives in the first components; noise is spread evenly over all 64. Keeping k ≈ 21 cuts the error from **9.3** to **5.6**.

PCA Before a Classifier



`StandardScaler → PCA(k) → LogisticRegression`, 5-fold CV on digits.

k = 2: **53%**. k = 15: **89%**. k = 30: **91%**. All 64: **92%**.

PCA ignores the labels: the directions of most variance need not be the directions that separate the classes.

PCA in scikit-learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)          # keep 95% of variance
Z_train = pca.fit_transform(X_train)  # learn μ and directions
Z_test = pca.transform(X_test)        # reuse them: no leakage

pca.n_components_                 # 29 on digits
pca.explained_variance_ratio_     # the scree plot
pca.components_                   # rows: eigendigits
X_back = pca.inverse_transform(Z_test)  # μ + Z Qₖᵀ
```

Integer or fraction: `n_components=10` keeps 10; `0.95` keeps enough for 95%.

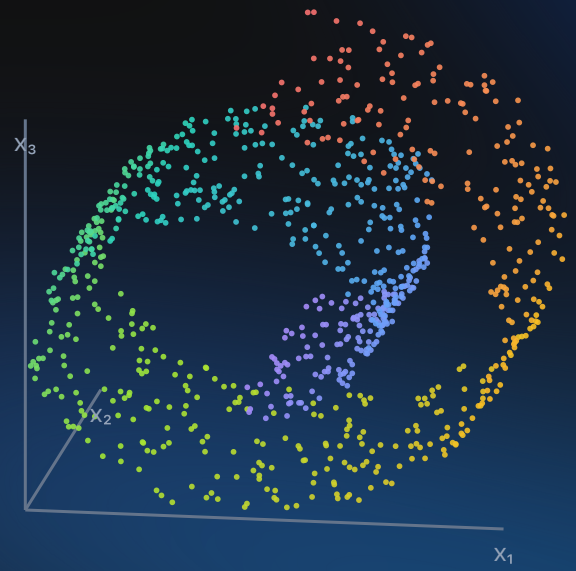
Fit on training data only: in a `Pipeline`, cross-validation refits PCA inside each fold.

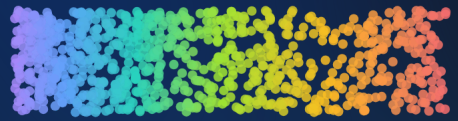
Different units? Put a `StandardScaler` in front.

Part 4 · When Flat Is Not Enough

Data can be low-dimensional and still not lie near any plane.

The Swiss Roll

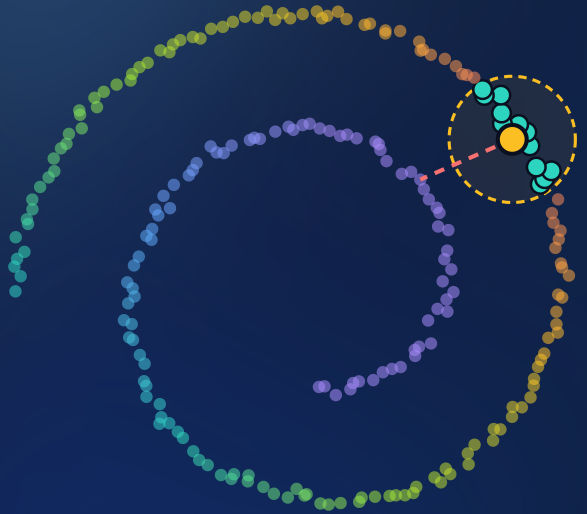




unrolled: the 2D sheet it really is

A **manifold**: a surface that is 2D up close, curled up in 3D. Its natural coordinates are not any linear projection.

Straight Lines Cut Through Empty Space



radius 0.45

inside the circle: 12

nearby along the curve: 12

other arm (a shortcut): 0

nearest other-arm point:
straight line 0.71 · along the curve 8.71

Click any point to move the source.

Trust Only Small Neighborhoods

k = 6

```
edges: 701
short circuits: 0
```

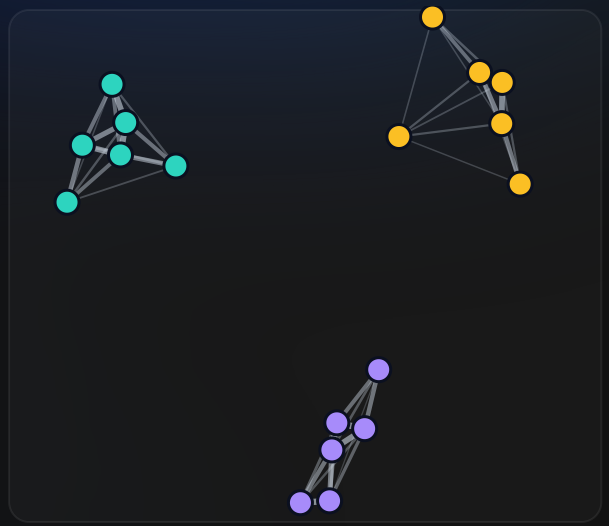
Too small: the graph falls apart into pieces. Too large: it bridges arms that are far apart on the curve.

Part 5 · t-SNE

t-distributed Stochastic Neighbor Embedding (van der Maaten & Hinton, 2008)

t-SNE in One Picture

original space: p_{ij}



2D map: q_{ij}

1 · p_{ij}: how strongly i and j are neighbors in the data (Gaussian)

2 · q_{ij}: the same, between their positions in the 2D map (Student-t)

3 · Move the map points until q matches p (KL divergence)

Step 1: Neighbor Probabilities

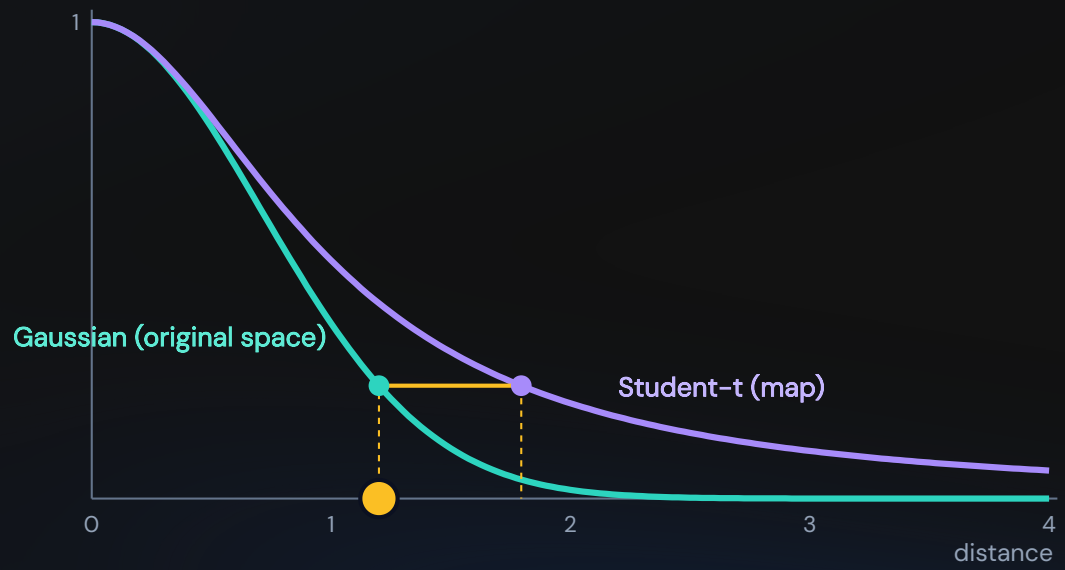
perplexity 10

σᵢ = 0.126
2^H(Pᵢ) = 10.0 ≈ effective neighbors
largest p_j|i = 0.241

Click a point in the tight cluster, then one in the diffuse cluster: same perplexity, very different σ.

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma_i^2)}, \qquad 2^{H(P_i)} = \text{perplexity}, \qquad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Step 2: A Heavy-Tailed Map



original distance 1.20 → same similarity at map distance 1.79(× 1.50)

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}$$

Crowding problem: in 64-D a point can have many equidistant neighbors; in 2D there isn't room. Heavy tails let moderately distant points move further out.

Step 3: Match with KL Divergence

$$C = \mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

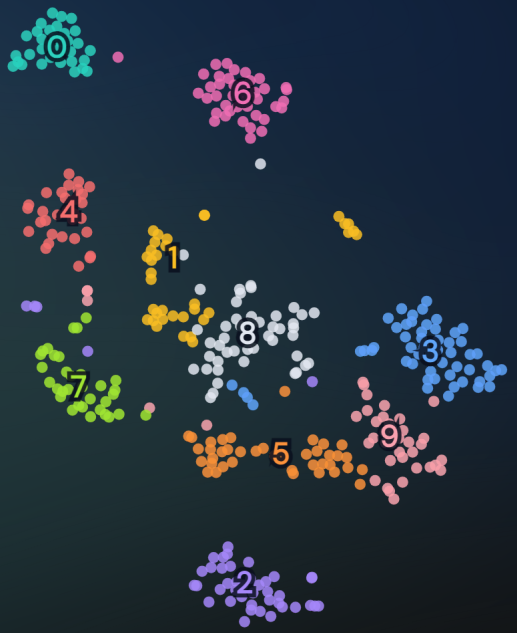
$$\frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j \underbrace{(p_{ij} - q_{ij})}_{\text{spring}} \frac{\mathbf{y}_i - \mathbf{y}_j}{1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2}$$

IN THE DATA	IN THE MAP	COST
neighbors (p large)	far (q small)	large
far (p small)	close (q large)	small

Tearing neighbors apart is punished; putting strangers together is barely noticed. t-SNE keeps **local** structure, not global.

Gradient descent with momentum, and **early exaggeration** (p × 12 for the first 250 steps) so clusters form first.

Watch t-SNE Run



iteration 1000 / 1000

done

KL(P||Q) = 0.436

↻ restart | perp 5 | perp 30 | perp 100

Same algorithm in Python ends at KL 0.436; sklearn's exact t-SNE on these points: 0.423.

Perplexity and Seeds Change the Picture

PERPLEXITY

2 | 5 | 15 | **30** | 50 | 100

PERPLEXITY 30, OTHER RANDOM SEEDS

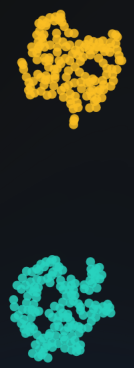
seed 1 | seed 2 | seed 3

Low: many tiny fragments; only the closest neighbors count.

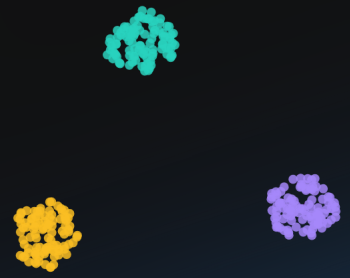
High: rounder, more global, clusters start to touch.

Seeds: same clusters, different arrangement.

How Not to Read a t-SNE Plot



Cluster sizes mean nothing
sd 1 vs sd 10: drawn the same size



Gaps may mean nothing
gaps of 10 and 50: drawn about equal

Noise can look clumpy
pure 30-D noise, perplexity 3

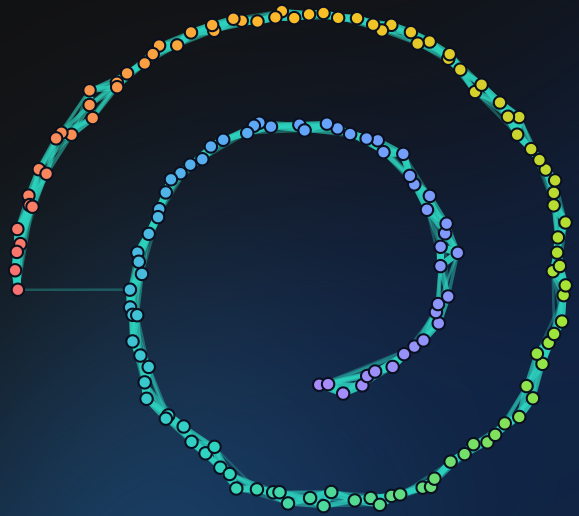
`t-SNE maps`

After Wattenberg, Viégas & Johnson, *How to Use t-SNE Effectively* (Distill, 2016). Use t-SNE to look and form hypotheses; confirm in the original space.

Part 6 · UMAP

Uniform Manifold Approximation and Projection (McInnes, Healy & Melville, 2018)

UMAP Step 1: A Fuzzy Neighbor Graph



n_neighbors 8

hover a point

Edge opacity = fuzzy weight w = a + b − ab of the two directed weights.

$$w_{j|i} = \exp\!\left(-\frac{d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i}{\sigma_i}\right) \ (j \text{ among the } k \text{ nearest}), \qquad w_{ij} = w_{j|i} + w_{i|j} - w_{j|i}\, w_{i|j}$$

UMAP Step 2: Lay Out the Graph

`start` `epoch 5` `epoch 15` `epoch 40` `epoch 100` `epoch 200`

Start: a spectral layout of the graph, not random.

Attract along edges; **repel** a few random non-neighbors (negative sampling).

$$\sum_{ij} w_{ij} \log \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}}$$

$v_{ij} = (1 + a\|y_i - y_j\|^{2b})^{-1}$, with a, b set by `min_dist`.

UMAP’s Two Knobs

N_NEIGHBORS · local ↔ global

5 | 15 | 50 | 200

MIN_DIST · tight clumps ↔ even spread

0 | 0.1 | 0.5 | 0.99

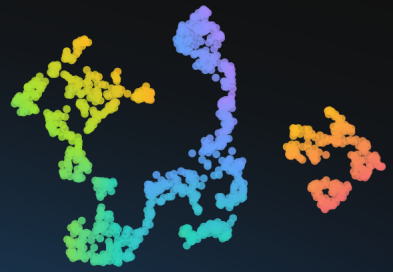
```
UMAP(n_neighbors=15, min_dist=0.1)
```

The Swiss Roll, Three Ways

PCA

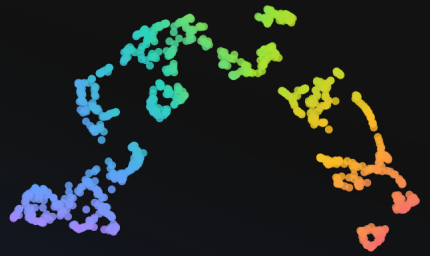
`trustworthiness 0.898`

t-SNE



`trustworthiness 0.999`

UMAP



`trustworthiness 0.999`

Trustworthiness: are each point's 10 nearest neighbors in the map also near it in the original space? 1 = all of them.

Same Digits, Three Maps

PCA

trust 0.830 · <1 ms

t-SNE

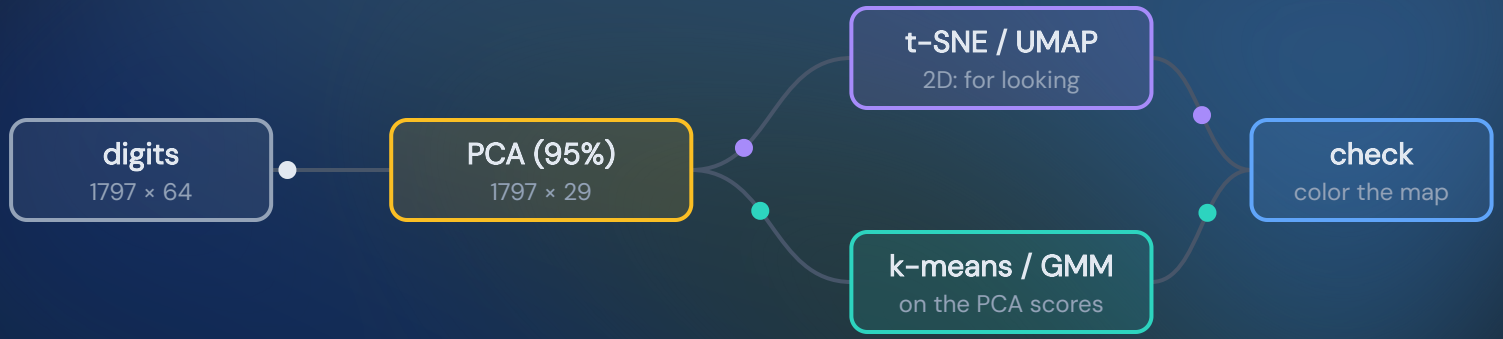
trust 0.993 · 1.0 s

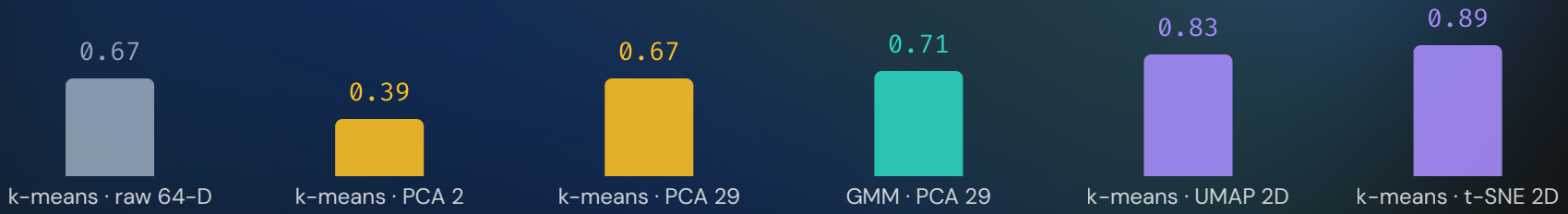
UMAP

trust 0.988 · 4.2 s

Hover any point: the same digit is ringed in all three maps. Trust = trustworthiness with 10 neighbors.

A Practical Pipeline





Adjusted Rand index between 10 clusters and the true digits (1 = perfect, 0 = chance)

t-SNE and UMAP in Code

```python
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap                                  # pip install umap-learn

pca = PCA(n_components=50).fit(X_train)
Z = pca.transform(X_train)                   # denoise and speed up first

E = TSNE(perplexity=30, init="pca",
         random_state=0).fit_transform(Z)  # no .transform() for new data

reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0)
E = reducer.fit_transform(Z)
E_new = reducer.transform(pca.transform(X_new))  # UMAP can embed new points
```

init="pca" (the default now): a more stable, more global starting layout than random.

Fix random_state to reproduce a map, then **vary** it to see what is stable.

Scale: t-SNE uses Barnes–Hut, O(n log n); UMAP handles millions.

Choosing a Method

	MAP	KEEPS	NEW POINTS	AXES MEAN	REPEATABLE	USE IT TO
PCA	linear	global variance	yes, exact	variance directions	yes	compress, denoise, preprocess, a first look
t-SNE	nonlinear	local neighbors	no	nothing	varies with seed	look at cluster structure
UMAP	nonlinear	local + some global	yes, approx.	nothing	varies with seed	look at large data; preprocessing with care

Never read **sizes, densities or distances between clusters** off a t-SNE or UMAP map.

Also worth knowing: **kernel PCA**, **Isomap**, **autoencoders** (next module).

Summary

Few real dimensions

Data with many columns often varies along a few directions; distances degrade in high d.

PCA = eigenvectors of Σ

Max variance = min reconstruction error. λ_j = variance kept by component j.

Use it well

Center (and scale); choose k by variance or CV; fit on training data only.

Manifolds need local views

Straight-line distance is only trustworthy between near neighbors: build a neighbor graph.

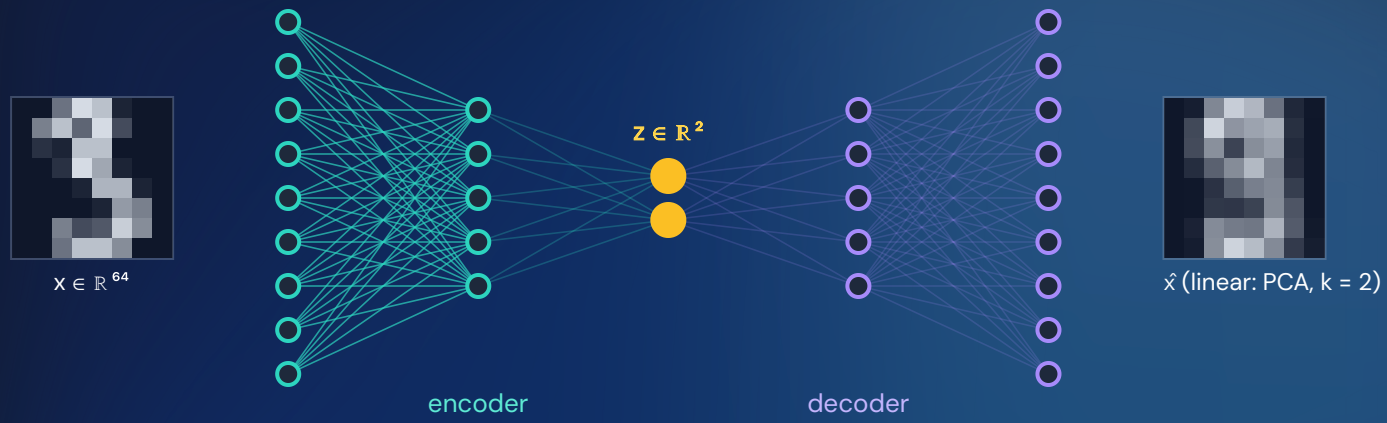
t-SNE & UMAP

Match neighbor probabilities between data and map. Perplexity / n_neighbors set the scale.

Read maps carefully

Sizes, gaps, and seeds can mislead. Confirm in the original space.

Next: Neural Networks



Next module: networks of simple units, trained by gradient descent. One of them, the **autoencoder**, squeezes data through a narrow middle layer and rebuilds it. With linear layers and squared error it learns **PCA's subspace**; with nonlinear layers it can follow curved manifolds.